

SUPPLEMENTARY MATERIAL

A. More Logic-Discrepancy Scenarios in Running Example

Scenario: Loading the Main Page. When users log in to the shopping mini-program, the main page displays the items they selected during their previous session. Figure 7 illustrates a logic discrepancy in cross-layer communications that arise when running the mini-program on different platforms (Android v.s. Windows). Specifically, the total shown on Android is \$58 (the sum of the selected items), whereas the total shown on Windows is \$116 (exactly twice the Android value). As a result, users may overpay at checkout, especially if they do not carefully verify the total amount.

Further inspection reveals that this discrepancy stems from inconsistent life-cycle management in the data-synchronization stage between Android and Windows. In particular, *shopping_info* is synchronized once during page loading on Android, triggering *updateTotal* only once. In contrast, on Windows, *shopping_info* is synchronized twice during page loading, causing *updateTotal* to be invoked twice and the total to be added twice.

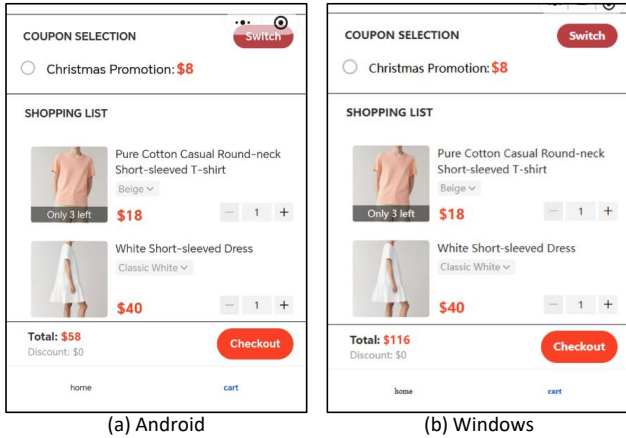


Fig. 7: Logic discrepancies when loading the main page.

Scenario: Selecting a Coupon. After the main page loads, users may select a coupon. Figure 8 illustrates a logic discrepancy in cross-layer communications that arise when running the mini-program under the GlassEasel+WebView (GW) configuration on different platforms (DevTools v.s. Android). Specifically, the discount shown on DevTools is \$0, whereas it is \$16 on Android (twice the selected coupon value). As a result, merchants may suffer financial losses due to excessive discounts.

Further inspection reveals that this discrepancy stems from two sources: inconsistent shadow-data maintenance during the data synchronization stage and inconsistent object (de)serialization during the object transmission stage. In particular, DevTools falls back from GW to EW, whereas Android remains in GW. Under EW, modifying an array element (*coupons*[1]) mistakenly causes the deserialized result to become an empty object, which explains why the discount becomes \$0 on DevTools. Under GW, updating *coupons*[1]

triggers shadow-data maintenance for *discount*, which incorrectly invokes *updateDiscount* twice, explaining why the discount becomes \$16 on Android.

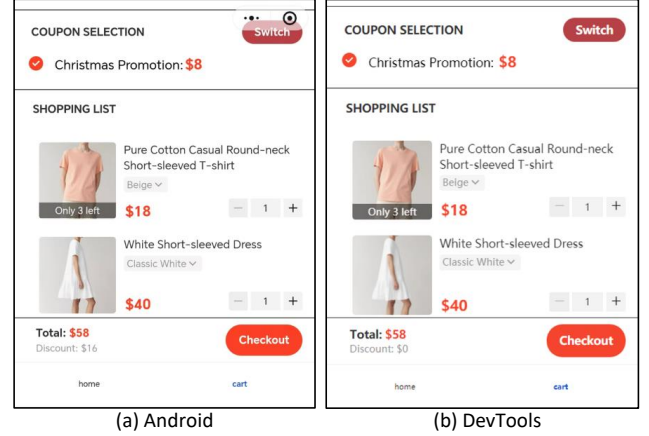


Fig. 8: Logic discrepancies when selecting a coupon.

B. Illustration of MINIDIFF's Workflow

Testcase Generation. Using the motivating example as a running case, we illustrate how a test case is generated in practice. This test case is composed of two simple subcases placed in different page elements. Although both subcases instantiate Channel ⑦, they have different sources and destinations and are therefore logically independent. For the first subcase, the selected building blocks are Channel ⑦, data type Object, page element <text>, and content position. For the second subcase, the selected building blocks are also Channel ⑦ and data type Object, but the page element is instantiated at the attribute position of <text>.

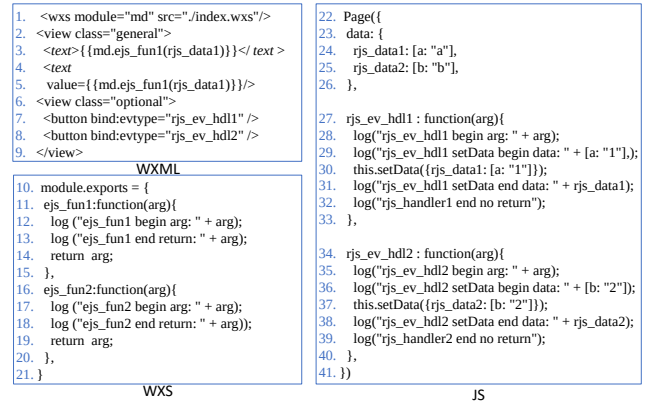


Fig. 9: Example of testcase.

The generated JS code contains two variables including *rjs_data1* and *rjs_data2*, and two event handlers including *rjs_ev_hdl1* and *rjs_ev_hdl2*, each of which invokes *setData* on its corresponding variable. The generated WXS code contains two functions related to Channel ⑦, including *ejs_fun1: function(arg)* and *ejs_fun2: function(arg)*. The generated WXML code contains two <text> elements and two

<button> elements. The <text> elements bind data using “*md.ejs_funX(rjs_dataX)*”, while the <button> elements are bound to the corresponding JS event handlers. The final generated code is shown in Figure 9.

Code Execution. We illustrate the execution process using the same motivating example. First, the generated test case is copied into a testing project, and *app.json* is adjusted to match the target configuration. The project is then compiled and executed on the corresponding platforms. Next, MINIDIFF invokes the Automator SDK to perform UI testing. Since the example contains no <input> fields and only interactive <button> elements, the interaction sequence consists of reloading and tapping. Specifically, after the mini-program is loaded, MINIDIFF performs one reload, alternately taps the two buttons for two rounds, and then performs a final reload. After each interaction, it exports the current UI as an XML snapshot, resulting in seven snapshots in total. Once all interactions are completed, execution logs are collected. In this example, when button 1 is tapped, both WXS functions are invoked under EW and GW, whereas only *ejs_fun1* is invoked under GS. All behaviors are recorded in the logs.

Discrepancy Analysis. Consider Channel ⑦ ($RJS \xrightarrow{var} EJS \xrightarrow{fun} XML$). When the parameter of an EJS function is derived from *setData*, the RJS function performing the corresponding *setData* operation is treated as its upstream point. When the parameter is derived from initialization, the upstream point is a lifecycle function. The return value of the EJS function is then consumed by the XML element as dynamic data. A *setData* operation is considered an upstream point only when the updated variable matches the parameter of the EJS function. For example, *setData(data1)* is upstream of *fun1(data1)*, but not of *fun2(data2)* or *fun3()*. For parameterless functions such as *fun3()*, an upstream point exists only during initialization through lifecycle functions; otherwise, no upstream point exists. Suppose horizontal analysis finds that the oracle at the entry of an EJS function is triggered twice in one environment but only once in another. Vertical analysis then starts from this point and classifies the symptom as an execution-frequency discrepancy. It next traces the upstream point, which may be an RJS function containing the related *setData* operation, a lifecycle function, or no upstream point at all, to infer the contributing factor.

C. Detailed Results of Previously Unknown Discrepancies

Tables II, III and IV report the detection results of MINIDIFF on WeChat, Alipay and TikTok, respectively. In §VI-C, we presented an example of Root Cause ①. In this section, we present examples of the remaining root causes.

② Inconsistent Life-Cycle Management. Page loading behavior may vary across different platforms. Listing 2 presents an example illustrating this issue (P12 of Table IV) on TikTok. In this example, a function invocation tracker is used to count the number of times the function *fun* is invoked on page loading. On DevTools, an additional data update operation is

performed after initialization, resulting in a displayed count of “1”; while on other platforms, the count is displayed as “2”.

Listing 2 Example of inconsistent life-cycle management.

```
1<text>{{md.fun()}}</text> |XML|
2<wxs module="md"> |EJS|
3  var cnt = 0;
4  module.exports.fun = function() {return ++cnt;}
5</wxs>
```

③ Inconsistent Object (De)Serialization. Serialization and deserialization occur during cross-layer data communication. Some configurations cannot effectively handle the serialization and deserialization of function data. Listing 3 presents a simplified example (C23 of Table III) on TikTok. It is generated by instantiating the $RJS \xrightarrow{var} XML$ channel (Channel ②), with a function variable in RJS as the source, and the content of the <text> element in XML as the destination. The observation oracle is page data. In this example, WR displays nothing on the page, whereas NR correctly displays “1”.

Listing 3 Example of inconsistent object (de)serialization.

```
1<text>{{fun()}}</text> |XML|
2data: {fun: function() {return '1'}} |RJS|
```

④ Confused Messaging Channel. EJS and RJS operate in separate scopes. When two variables (one in EJS and one in RJS) share the same name, the priority of the messaging channel determines whether a variable reference resolves to the EJS or RJS variable. On WeChat, EW prioritizes the EJS scope, whereas GW and GS prioritize the RJS scope. Listing 4 presents a simplified example (C10 of Table II). It is generated through the conflicting combination of instantiations of the $RJS \xrightarrow{var} XML$ channel (Channel ①) and the $EJS \xrightarrow{var} XML$ channel (Channel ⑤). In this example, a EJS module shares the same name “*md*” with the variable specified in the *wx:for-item* directive, which refers to a RJS list. In EW, “{{*md.key*}}” refers to the “*key*” defined in EJS, with a value of “*wxs_list_value*”. In GW and GS, “{{*md.key*}}” refers to the “*key*” item of “*var_list*” defined in RJS, with a value of “*js_list_value*”.

Listing 4 Example of confuse messaging channel.

```
1<view wx:for="{{var_list}}" wx:for-item="md"> |XML|
2  <text>{{md.key}}</text>
3</view>
4<wxs module="md"> |EJS|
5  module.exports = {key: "wxs_list_value"}
6</wxs>
7data: {var_list: [{key: "js_list_value"}]} |RJS|
```

⑤ Inconsistent Virtual DOM Diffing. Different configurations implement different virtual DOM diffing algorithms. On WeChat, EW and GW use coarse-grained diffing, while GS uses fine-grained diffing. Listing 5 presents a simplified example (C17 of Table II). It is generated through the coexisting combination of two instantiations of the $RJS \xrightarrow{var} EJS \xrightarrow{fun} XML$ channel (Channel ⑦). Specifically, these two instances coexist within a testcase, each with distinct sources and destinations. In this example, tapping the button triggers an update to *var1*, which should only affect the re-rendering of the <text> element with id “text1”, as correctly handled in GS. However,

in EW and GW, the `<text>` element with id “text2” is also re-rendered. The re-rendering behavior can be observed through function invocation trackers. Therefore, the content of the `<text>` element with id “text2” is “1” in GS, but “2” in EW and GW.

Listing 5 Example of inconsistent virtual DOM diffing.

```

1<text id="text1">{{md.fun1(var1)}}</text> |XML|
2<text id="text2">{{md.fun2(var2)}}</text>
3<button bind:tap="tap_handler"></button>
4<wxs module="md"> |EJS|
5  var cnt1 = 0, cnt2 = 0;
6  module.exports = {
7    fun1: function(data) {return ++cnt1;}
8    fun2: function(data) {return ++cnt2;}
9  }
8</wxs>
9data: {var1: 1, var2: 2} |RJS|
10tap_handler: function() {this.setData({var1: 0});}

```

⑥ Inconsistent Thread Model. On Windows, The WeChat mini-program framework does not follow the typical two-thread model seen on other platforms. Instead, both layers operate within a single thread. This design choice can introduce discrepancies in program behaviors. Listing 6 presents a simplified example illustrating this issue (P8 of Table II). In this example, the oracle shows function execution order. After clicking the button, the log output on the Windows platform shows “js_fun begin”, “wxs_fun”, and “js_fun end” in sequence, indicating that the `setData` operation and the UI update are performed synchronously. On other platforms, the log output shows “js_fun begin”, “js_fun end”, and “wxs_fun” in sequence, indicating that the logic layer’s processing and the rendering layer’s UI update are executed asynchronously.

Listing 6 Example of inconsistent thread model.

```

1<text>{{md.fun(var_num)}}</text> |XML|
2<button bind:tap="tap_handler"></button>
3<wxs module="md"> |EJS|
4  module.exports.fun = function(data) {log("wxs_fun"); ...}
5</wxs>
6data: {var_num: 0} |RJS|
7tap_handler: function() {
8  log("js_fun begin");
9  this.setData({var_num: 0});
10 log("js_fun end");
11 }

```

TABLE II
ZERO-DAY LOGIC DISCREPANCIES IN CROSS-LAYER COMMUNICATIONS OF WECHAT.

Issue	Discrepancies			Data Channel	Data Type	Page Element	Page Position	Observation Oracle	Combination	UI Interaction	Root Cause	Status	MINIDIFF	O ⁻	C ⁻	U ⁻
																
C1	GWtEWGS	EWtGW	N/A	⑦	boolean	*	wx:if	page data	none	button tapping	①	●	✓	✓	✓	✓
C2	GWtEWGS	EWtGW	N/A	⑥	boolean	*	wx:if	page data	none	button tapping	①	●	✓	✓	✓	✓
C3	GWtEWGS	EWtGW	N/A	⑦	*	*	*	execution frequency	none	button tapping	①	●	✓	✓	✓	✓
C4	GStEWGW	N/A	EWtGS	⑥	*	*	*	execution frequency	none	reloading	①	○	✓	✓	✓	✓
C5	GStEWGW	N/A	EWtGS	⑦	*	*	*	execution frequency	none	reloading	①	○	✓	✓	✓	✓
C6	EWtGWGS	EWtGW	EWtGS	③	array	*	content	page data	none	button tapping	③	●	✓	✓	✓	✓
C7	EWtGWGS	EWtGW	EWtGS	③	array	*	attribute	page data	none	button tapping	③	●	✓	✓	✓	✓
C8	EWtGWGS	EWtGW	EWtGS	⑦	array	*	attribute	variable values	none	button tapping	③	●	✓	✓	✓	✓
C9	GStEWGW	N/A	EWtGS	②	function	*	*	execution frequency	none	none	③	○	✓	✓	✓	✓
C10	EWtGWGS	EWtGW	EWtGS	①	array	*	wx:for-item	page data	conflicting	none	④	○	✓	✓	✓	✓
C11	EWtGWGS	EWtGW	EWtGS	②	array	*	wx:for-item	page data	conflicting	none	④	○	✓	✓	✓	✓
C12	EWtGWGS	EWtGW	EWtGS	③	array	*	wx:for-item	page data	conflicting	button tapping	④	○	✓	✓	✓	✓
C13	EWtGWGS	EWtGW	EWtGS	⑤	array	*	wx:for-item	page data	conflicting	none	④	○	✓	✓	✓	✓
C14	EWtGWGS	EWtGW	EWtGS	⑥	array	*	wx:for-item	page data	conflicting	none	④	○	✓	✓	✓	✓
C15	EWtGWGS	EWtGW	EWtGS	⑦	array	*	wx:for-item	page data	conflicting	button tapping	④	○	✓	✓	✓	✓
C16	GStEWtGW	N/A	EWtGS	⑦	*	*	*	execution frequency	coexisting (data)	button tapping	⑤	●	✓	✓	✓	✓
C17	GStEWGW	N/A	EWtGS	⑥ ⑦	*	*	*	execution frequency	coexisting (all)	button tapping	⑤	○	✓	✓	✓	✓
C18	GStEWGW	N/A	EWtGS	③ ⑥	*	*	*	execution frequency	coexisting (all)	button tapping	⑤	○	✓	✓	✓	✓
C19	GStEWGW	N/A	EWtGS	④ ⑥	*	input	*	execution frequency	coexisting (all)	text entering	⑤	○	✓	✓	✓	✓
C20	GStEWGW	N/A	EWtGS	⑦	*	*	*	execution frequency	coexisting (element)	button tapping	⑤	○	✓	✓	✓	✓

Status: ○ for confirmed but not fixed, ● for partially fixed, ● for fully fixed.

TABLE III
ZERO-DAY LOGIC DISCREPANCIES IN CROSS-LAYER COMMUNICATIONS OF ALIPAY.

Issue	Discrepancies			Data Channel	Data Type	Page Element	Page Position	Observation Oracle	Combination	UI Interaction	Root Cause	Status	MINIDIFF	O ⁻	C ⁻	U ⁻
																
P1			N/A	⑥	*	*	content	execution frequency	none	none	②	○	✓	✓	✓	✓
P2			N/A	⑦	*	*	content	execution frequency	none	none	②	○	✓	✓	✓	✓
P3	N/A			①	*	*	*	page data	none	none	③	●	✓	✓	✓	✓
P4	N/A			⑤	*	*	*	page data	none	none	③	●	✓	✓	✓	✓
P5	N/A	N/A		⑨	*	*	attribute	variable values	none	button tapping	③	●	✓	✓	✓	✓
P6			N/A	③ ⑥	*	*	*	execution frequency	coexisting (all)	none	⑤	●	✓	✓	✓	✓
P7			N/A	③ ⑦	*	*	*	execution frequency	coexisting (all)	none	⑤	●	✓	✓	✓	✓
P8			N/A	⑦	*	*	*	execution order	none	button tapping	⑥	○	✓	✓	✓	✓

Issue	Discrepancies			Data Channel	Data Type	Page Element	Page Position	Observation Oracle	Combination	UI Interaction	Root Cause	Status	MINIDIFF	O ⁻	C ⁻	U ⁻
																
C21	WRtNR	⑥	*	*	*	execution frequency	none	reloading	①	○	✓	✓	✓	✓	✓	✓
C22	WRtNR	⑦	*	*	*	execution frequency	none	reloading	①	○	✓	✓	✓	✓	✓	✓
C23	WRtNR	①	function	*	content	page data	none	none	③	○	✓	✓	✓	✓	✓	✓
C24	WRtNR	②	function	*	*	page data	none	none	③	○	✓	✓	✓	✓	✓	✓
C25	WRtNR	③	function	*	content	page data	none	button tapping	③	○	✓	✓	✓	✓	✓	✓
C26	WRtNR	⑤	function	*	content	page data	none	none	③	○	✓	✓	✓	✓	✓	✓
C27	WRtNR	⑨	*	*	attribute	variable values	none	button tapping	③	○	✓	✓	✓	✓	✓	✓

Issue	Discrepancies			Data Channel	Data Type	Page Element	Page Position	Observation Oracle	Combination	UI Interaction	Root Cause	Status	MINIDIFF	O ⁻	C ⁻	U ⁻
																
P9			①	array	*	attribute	page data	none	none	③	○	✓	✓	✓	✓	✓
P10			③	array	*	attribute	page data	none	button tapping	③	○	✓	✓	✓	✓	✓
P11			⑥	*	*	*	variable values	none	button tapping	③	●	✓	✓	✓	✓	✓

Status: ○ for confirmed but not fixed, ● for partially fixed, ● for fully fixed.

TABLE IV
ZERO-DAY LOGIC DISCREPANCIES IN CROSS-LAYER COMMUNICATIONS OF TIKTOK.

Issue	Discrepancies			Data Channel	Data Type	Page Element	Page Position	Observation Oracle	Combination	UI Interaction	Root Cause	Status	MINIDIFF	O ⁻	C ⁻	U ⁻
																
P12			⑥	*	*	content	execution frequency	none	none	②	●	✓	✓	✓	✓	✓
P13			⑦	*	*	content	execution frequency	none	none	②	●	✓	✓	✓	✓	✓
P14			①	array	*	a:for	page data	none	none	③	●	✓	✓	✓	✓	✓
P15			③	array	*	a:for	page data	none	none	③	●	✓	✓	✓	✓	✓
P16			⑤	array	*	a:for	page data	none	none	③	●	✓	✓	✓	✓	✓
P17			③ ⑥	*	*	execution frequency	coexisting (all)	none	none	③	●	✓	✓	✓	✓	✓
P18			③ ⑦	*	*	execution frequency	coexisting (all)	none	none	③	●	✓	✓	✓	✓	✓

Status: ○ for confirmed but not fixed, ● for partially fixed, ● for fully fixed.